

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний аерокосмічний університет

«Харківський авіаційний інститут»

Факультет систем управління літальних апаратів

Кафедра систем управління літальних апаратів

Лабораторна робота №8

з дисципліни «Алгоритмізація та програмування»

(назва дисципліни)

на тему: «Реалізація алгоритмів обробки двовимірних масивів мовою C++»

XAI.301.312.G7.3.LP

Виконав: здобувач 1 курсу, групи 312

напряму підготовки (спеціальності)

G7 «Автоматизація комп'ютерно-інтегровані
технології та робототехніка»

ОПП Інженерія мобільних додатків

(шифр і назва напряму підготовки (спеціальності))

С. С. БОЯРСЬКИЙ

(прізвище й ініціали студента)

Прийняв: доцент каф.301, канд. техн. наук,

О. В. ГАВРИЛЕНКО

(посада, науковий ступінь, прізвище й ініціали)

Харків - 2026

МЕТА РОБОТИ

Вивчити теоретичний матеріал по алгоритмам обробки масивів на мові C++, а також бібліотеки для роботи з файлами і реалізувати оголошення, введення з файлу, обробку і виведення в файл одновимірних і двовимірних масивів на мові C++ в середовищі Visual Studio / QtCreator.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1. За допомогою текстового редактору створити текстовий файл «array_in_97.txt» з елементами вихідного масиву. У програмі на C++ зчитати і перетворити цей масив (видалити з масиву всі однакові елементи, залишивши їх останні входження), ім'я файлу ввести з консолі. Вивести результати у файл «array_out_97.txt».

Завдання 2. За допомогою текстового редактору створити текстовий файл «matr_in_19.txt» з елементами вихідного двовимірного масиву. Зчитати і обробити матрицю (знайти суму елементів кожного її рядка), ім'я файлу ввести з консолі. Дописати результати в той же файл.

Завдання 3. Вивчити метод сортування "Вибір" (Сортування вибором за зростанням для цілих чисел). Проаналізувати його складність і продемонструвати на прикладі з 7-ми елементів. Зчитування і виведення відсортованого масиву організувати на файлах.

Завдання 4. Структурувати проєкт програми у декілька файлів: main.cpp, array_utils.h, array_utils.cpp, matrix_utils.h, matrix_utils.cpp.

Завдання 5. Використовуючи засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок.

ВИКОНАННЯ РОБОТИ

Завдання 1

Вхідні дані: MAX_SIZE — ціле число, константа (максимальний розмір масиву N=50). n — ціле число (int), фактична кількість елементів у масиві (від 2 до 50). arr — масив цілих чисел розміром n. in_file — рядок (string), ім'я вхідного текстового файлу.

Вихідні дані: Вихідний масив до змін, виведений у консоль. Змінений масив (без дублікатів із збереженням останніх входжень), виведений у консоль та записаний у текстовий файл «array_out_97.txt».

Алгоритм зчитування та запису: Для отримання даних створюється об'єкт файлового потоку введення ifstream. З файлу спочатку зчитується одне ціле число, яке визначає логічну довжину масиву n. Потім за допомогою циклу for виконується послідовне читання n елементів у масив. Після завершення операції потік закривається. Для збереження результатів створюється об'єкт класу ofstream. У файл спочатку записується нове значення n, а потім у циклі через пробіл виводяться всі елементи модифікованого масиву.

```
--- МЕНЮ ---
1. Завдання 1 (Array97 - видалення дублікатів)
2. Завдання 2 (Matrix19 - сума рядків)
3. Завдання 3 (Сортування вибором)
0. Вихід
Ваш вибір: 1
Введіть ім'я вхідного файлу (або enter для array_in_97.txt):
Вихідний масив: 10 20 10 30 40 20 50 10
Масив після обробки: 30 40 20 50 10
Результат записано у файл array_out_97.txt
```

Рисунок 1 – Результат виконання завдання 1

Завдання 2

Вхідні дані: MAX_ROWS, MAX_COLS — цілі числа, константи обмеження матриці (20x20). m — кількість рядків (int), n — кількість стовпців (int). mx — двовимірний масив дійсних чисел (double) розміром m на n. in_file — рядок (string), ім'я файлу.

Вихідні дані: Вихідна матриця, відображена в консолі. Одновимірний масив sums розміром m, що містить суми елементів для кожного рядка матриці. Результати, які дописуються в кінець файлу «matr_in_19.txt».

Алгоритм зчитування та допису (вербальний опис): Зчитування двовимірного масиву реалізовано за допомогою об'єкта ifstream. Перші два значення у файлі інтерпретуються як кількість рядків m та стовпців n. Далі за допомогою вкладених циклів for зчитуються елементи матриці за рядками. Обчислені суми записуються в той самий файл за допомогою об'єкта ofstream, який ініціалізується з параметром ios::app (append). Це забезпечує збереження початкових даних файлу та додавання нових результатів у його кінець.

```
--- МЕНЮ ---
1. Завдання 1 (Array97 - видалення дублікатів)
2. Завдання 2 (Matrix19 - сума рядків)
3. Завдання 3 (Сортування вибором)
0. Вихід
Ваш вибір: 2
Введіть ім'я файлу (enter для matr_in_19.txt):
Вихідна матриця:
1.5      2      3.5      1
4         5         6         2
7.5      8         9.5      3
Результати дописано у файл matr_in_19.txt
```

Рисунок 2 – Результат виконання завдання 2

Завдання 3

Вхідні дані: n — кількість елементів масиву (int). arr — масив цілих чисел (int) розміром n , зчитаний з файлу «sort_in_3.txt».

Вихідні дані: Відсортований за зростанням масив цілих чисел, записаний у файл «sort_out_3.txt» та виведений на екран.

Аналіз складності: Часова складність алгоритму сортування вибором становить $O(N^2)$ в усіх випадках (найгірший, середній, найкращий), оскільки кількість порівнянь завжди залишається незмінною через необхідність повного перегляду невідсортованої частини масиву для пошуку мінімального елемента. Складність за пам'яттю — $O(1)$, оскільки сортування виконується без використання додаткових масивів.

Покроковий приклад для 7 елементів (29, 10, 14, 37, 13, 25, 8):

- Крок 1: Мінімум на проміжку $[0...6]$ це 8 (індекс 6). Обмін з елементом на індексі 0 (29). Масив: [8, 10, 14, 37, 13, 25, 29]
- Крок 2: Мінімум на проміжку $[1...6]$ це 10 (індекс 1). Обмін не потрібен. Масив: [8, 10, 14, 37, 13, 25, 29]
- Крок 3: Мінімум на проміжку $[2...6]$ це 13 (індекс 4). Обмін з елементом на індексі 2 (14). Масив: [8, 10, 13, 37, 14, 25, 29]
- Крок 4: Мінімум на проміжку $[3...6]$ це 14 (індекс 4). Обмін з елементом на індексі 3 (37). Масив: [8, 10, 13, 14, 37, 25, 29]
- Крок 5: Мінімум на проміжку $[4...6]$ це 25 (індекс 5). Обмін з елементом на індексі 4 (37). Масив: [8, 10, 13, 14, 25, 37, 29]
- Крок 6: Мінімум на проміжку $[5...6]$ це 29 (індекс 6). Обмін з елементом на індексі 5 (37). Масив: [8, 10, 13, 14, 25, 29, 37]

void read_array_from_file(string filename, int arr[], int &n)

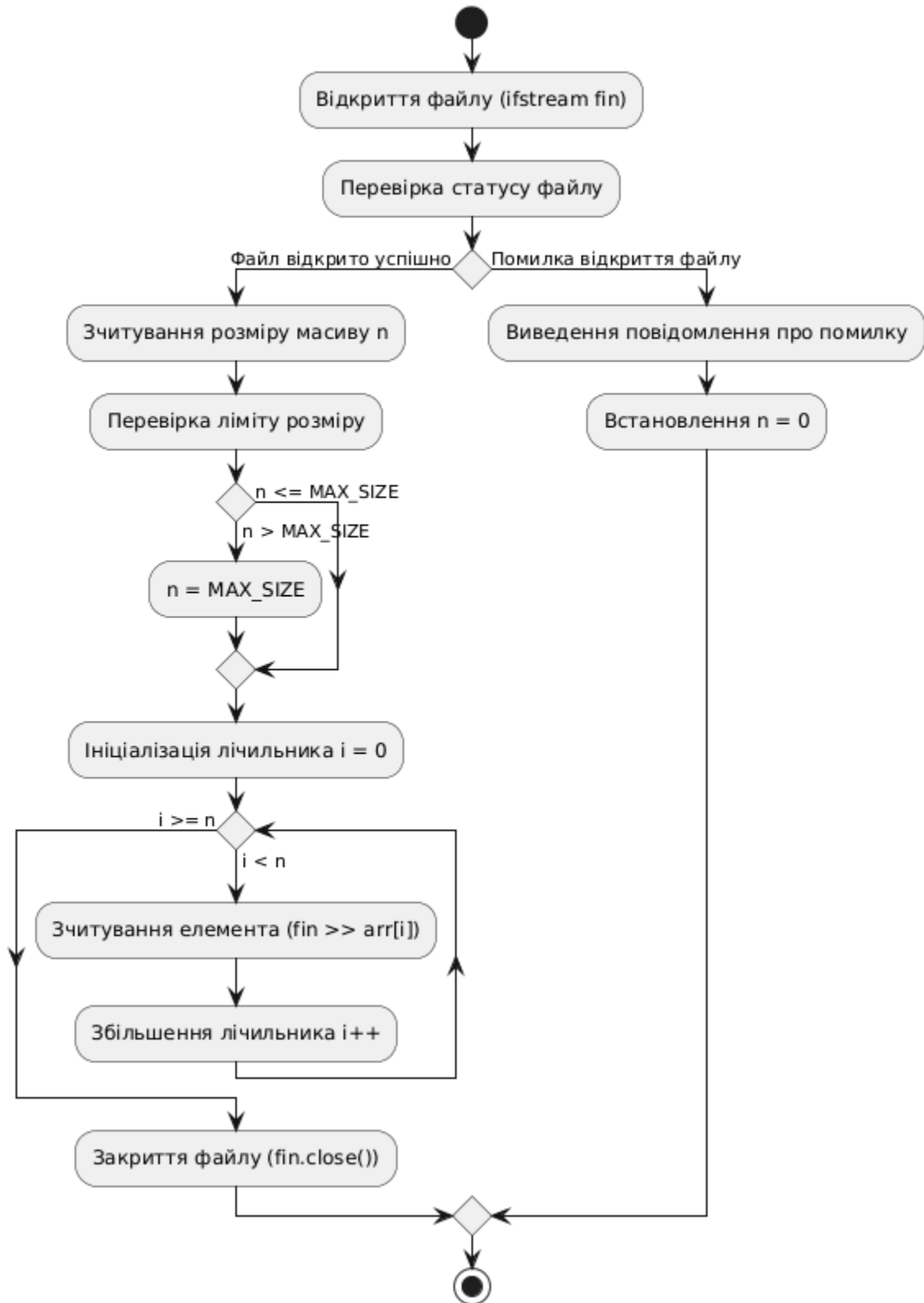


Рисунок 3 – Діаграма активності функції зчитування з файлу

void selection_sort(int arr[], int n)

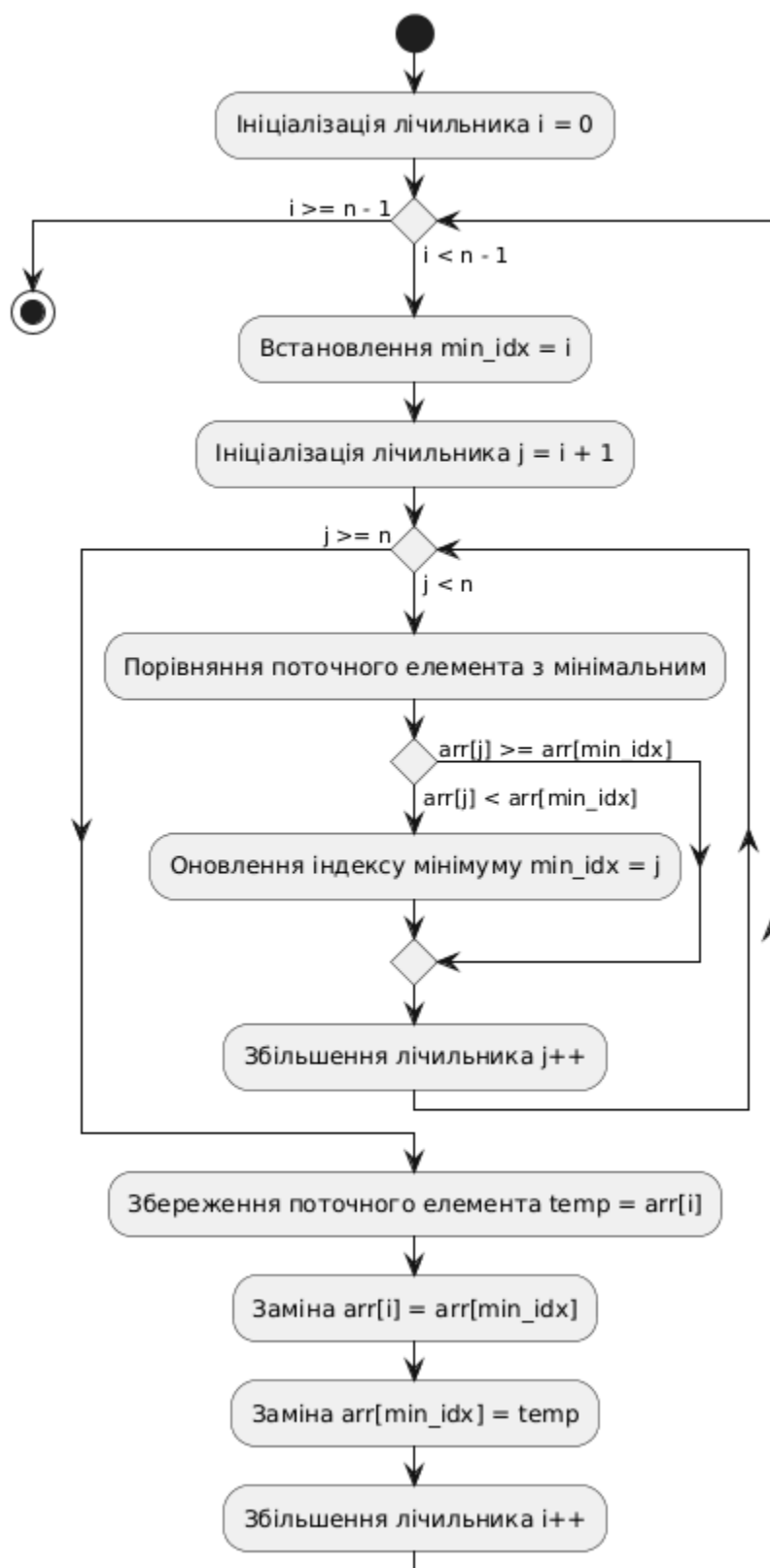


Рисунок 4 – Діаграма активності функції сортування вибором

```
--- МЕНЮ ---  
1. Завдання 1 (Array97 - видалення дублікатів)  
2. Завдання 2 (Matrix19 - сума рядків)  
3. Завдання 3 (Сортування вибором)  
0. Вихід  
Ваш вибір: 3  
Несортований масив: 29 10 14 37 13 25 8  
Відсортований масив: 8 10 13 14 25 29 37  
Записано у sort_out_3.txt
```

Рисунок 5 – Результат виконання завдання 3

ВИСНОВКИ

У ході виконання лабораторної роботи було успішно застосовано теоретичні знання з алгоритмів сортування (метод вибору) та роботи з файловими потоками (ifstream / ofstream) у мові C++. Програму було структуровано на кілька модулів (заголовні файли та файли реалізації згідно з принципами інженерії ПЗ), що полегшує її підтримку. Завдання виконано з дотриманням усіх вимог методичних вказівок.

ДОДАТОК А

ЛІСТИНГ КОДУ ПРОГРАМИ

Файл: main.cpp

```
#include <iostream>
#include <string>
#include "array_utils.h"
#include "matrix_utils.h"

using namespace std;

int main() {
    int choice;
    do {
        cout << "\nМЕНЮ" << endl;
        cout << "1. Завдання 1 (Array97 - видалення дублікатів)" << endl;
        cout << "2. Завдання 2 (Matrix19 - сума рядків)" << endl;
        cout << "3. Завдання 3 (Сортування вибором)" << endl;
        cout << "0. Вихід" << endl;
        cout << "Ваш вибір: ";
        cin >> choice;

        if (choice == 1) {
            task_array_97();
        } else if (choice == 2) {
            task_matrix_19();
        } else if (choice == 3) {
            task_sort_3();
        }
    } while (choice != 0);

    return 0;
}
```

Файл: array_utils.h

```
#ifndef ARRAY_UTILS_H
#define ARRAY_UTILS_H

#include <string>

const int MAX_SIZE = 50;

void task_array_97();
void task_sort_3();

void read_array_from_file(std::string filename, int arr[], int &n);
void write_array_to_file(std::string filename, const int arr[], int n);
void print_array(const int arr[], int n);

void remove_duplicates(int arr[], int &n);
void selection_sort(int arr[], int n);
```

```
#endif
```

Файл: array_utils.cpp

```
#include "array_utils.h"
#include <iostream>
#include <fstream>

using namespace std;

void task_array_97() {
    int arr[MAX_SIZE];
    int n = 0;
    string in_file = "array_in_97.txt";
    string out_file = "array_out_97.txt";

    cout << "Введіть ім'я вхідного файлу (або enter для array_in_97.txt): ";
    string temp;
    cin.ignore();
    getline(cin, temp);
    if (!temp.empty()) in_file = temp;

    read_array_from_file(in_file, arr, n);
    if (n == 0) return;

    cout << "Вихідний масив: ";
    print_array(arr, n);

    remove_duplicates(arr, n);

    cout << "Масив після обробки: ";
    print_array(arr, n);

    write_array_to_file(out_file, arr, n);
    cout << "Результат записано у файл " << out_file << endl;
}

void task_sort_3() {
    int arr[MAX_SIZE];
    int n = 0;
    string in_file = "sort_in_3.txt";
    string out_file = "sort_out_3.txt";

    read_array_from_file(in_file, arr, n);
    if (n == 0) return;

    cout << "Несортований масив: ";
    print_array(arr, n);

    selection_sort(arr, n);

    cout << "Відсортований масив: ";
```

```

    print_array(arr, n);

    write_array_to_file(out_file, arr, n);
    cout << "Записано у " << out_file << endl;
}

void remove_duplicates(int arr[], int &n) {
    for (int i = 0; i < n; i++) {
        bool has_duplicate_later = false;
        for (int j = i + 1; j < n; j++) {
            if (arr[i] == arr[j]) {
                has_duplicate_later = true;
                break;
            }
        }
        if (has_duplicate_later) {
            for (int k = i; k < n - 1; k++) {
                arr[k] = arr[k + 1];
            }
            n--;
            i--;
        }
    }
}

void selection_sort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min_idx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx]) {
                min_idx = j;
            }
        }
        int temp = arr[i];
        arr[i] = arr[min_idx];
        arr[min_idx] = temp;
    }
}

void read_array_from_file(string filename, int arr[], int &n) {
    ifstream fin(filename);
    if (!fin.is_open()) {
        cout << "Помилка відкриття файлу " << filename << endl;
        n = 0;
        return;
    }
    fin >> n;
    if (n > MAX_SIZE) n = MAX_SIZE;
    for (int i = 0; i < n; i++) {
        fin >> arr[i];
    }
    fin.close();
}

void write_array_to_file(string filename, const int arr[], int n) {

```

```

    ofstream fout(filename);
    if (!fout.is_open()) {
        cout << "Помилка запису у файл " << filename << endl;
        return;
    }
    fout << n << endl;
    for (int i = 0; i < n; i++) {
        fout << arr[i] << " ";
    }
    fout << endl;
    fout.close();
}

void print_array(const int arr[], int n) {
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

```

Файл: matrix_utils.h

```

#ifndef MATRIX_UTILS_H
#define MATRIX_UTILS_H

#include <string>

const int MAX_ROWS = 20;
const int MAX_COLS = 20;

void task_matrix_19();

void read_matrix_from_file(std::string filename, double mx[][MAX_COLS], int &m, int &n);
void append_matrix_results_to_file(std::string filename, const double sums[], int m);
void print_matrix(const double mx[][MAX_COLS], int m, int n);
void calculate_row_sums(const double mx[][MAX_COLS], int m, int n, double sums[]);

#endif

```

Файл: matrix_utils.cpp

```

#include "matrix_utils.h"
#include <iostream>
#include <fstream>

using namespace std;

void task_matrix_19() {
    double mx[MAX_ROWS][MAX_COLS];
    double sums[MAX_ROWS];
    int m = 0, n = 0;
    string in_file = "matr_in_19.txt";
}

```

```

    cout << "Введіть ім'я файлу (enter для matr_in_19.txt): ";
    string temp;
    cin.ignore();
    getline(cin, temp);
    if (!temp.empty()) in_file = temp;

    read_matrix_from_file(in_file, mx, m, n);
    if (m == 0 || n == 0) return;

    cout << "Вихідна матриця:" << endl;
    print_matrix(mx, m, n);

    calculate_row_sums(mx, m, n, sums);

    append_matrix_results_to_file(in_file, sums, m);
    cout << "Результати дописано у файл " << in_file << endl;
}

void calculate_row_sums(const double mx[][MAX_COLS], int m, int n, double sums[]) {
    for (int i = 0; i < m; i++) {
        sums[i] = 0;
        for (int j = 0; j < n; j++) {
            sums[i] += mx[i][j];
        }
    }
}

void read_matrix_from_file(string filename, double mx[][MAX_COLS], int &m, int &n) {
    ifstream fin(filename);
    if (!fin.is_open()) {
        cout << "Помилка відкриття файлу " << filename << endl;
        m = 0; n = 0;
        return;
    }
    fin >> m >> n;
    if (m > MAX_ROWS) m = MAX_ROWS;
    if (n > MAX_COLS) n = MAX_COLS;
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            fin >> mx[i][j];
        }
    }
    fin.close();
}

void append_matrix_results_to_file(string filename, const double sums[], int m) {
    ofstream fout(filename, ios::app);
    if (!fout.is_open()) {
        cout << "Помилка дозапису " << filename << endl;
        return;
    }
    fout << endl << "Суми елементів рядків:" << endl;
    for (int i = 0; i < m; i++) {
        fout << sums[i] << " ";
    }
}

```

```

    }
    fout << endl;
    fout.close();
}

void print_matrix(const double mx[][MAX_COLS], int m, int n) {
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            cout << mx[i][j] << "t";
        }
        cout << endl;
    }
}
}

```

ДОДАТОК В ДІАЛОГ З ШІ ДЛЯ САМОАНАЛІЗУ

Я: Ти — викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання середнього рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей про файли та сортування.

ШІ: Добре, Сергію. Ось твої запитання для захисту лабораторної роботи №8.

Тестові питання:

1. Який клас у бібліотеці <fstream> використовується виключно для зчитування даних з файлу?

- А) ofstream
- Б) fstream
- В) ifstream
- Г) readstream

2. Який прапорець потрібно використати при відкритті файлу, щоб нові дані дописувалися в кінець (як у завданні 2)?

- А) ios::in
- Б) ios::out
- В) ios::trunc
- Г) ios::app

3. Що робить метод .is_open()?

- А) Відкриває файл.
- Б) Перевіряє, чи файл успішно відкрито та готовий до роботи.
- В) Закриває файл.
- Г) Зчитує перший рядок.

4. Яка часова складність сортування вибором (Selection Sort) у найгіршому випадку?

- А) $O(N \log N)$

- Б) $O(N)$
- В) $O(N^2)$
- Г) $O(1)$

5. Чому при передачі двовимірного масиву у функцію (наприклад, `double mx[][MAX_COLS]`) ми обов'язково вказуємо другий розмір?

- А) Щоб компілятор знав, скільки всього пам'яті виділяти.
- Б) Тому що без цього компілятор не зможе обчислити зсув адреси для переходу на наступний рядок.
- В) Це просто правило синтаксису, без практичного змісту.
- Г) Щоб уникнено помилок переповнення буфера.

Відкриті питання:

1. Поясни своїми словами, чому функція `main` має бути визначена найпершою в коді (або як ти забезпечив її працездатність, якщо інші функції написані нижче)?
2. Що станеться у завданні 1, якщо вхідний файл не існує, і як програма реагує на це згідно з твоїм кодом?
3. Чому в завданні 2, при підрахунку суми `sums[i] += mx[i][j]`, важливо обнуляти `sums[i]` перед внутрішнім циклом?
4. Як працює прапорець `ios::app` при роботі з `ofstream` і чим він відрізняється від поведінки за замовчуванням?
5. Поясни алгоритм видалення дублікатів із завдання 1. Як саме зміщуються елементи і чому ми зменшуємо розмірність `n`?

Я: Мої відповіді.

Тести: 1-В, 2-Г, 3-Б, 4-Б, 5-Б.

Відкриті:

1. Тому що вчитель вимагає, щоб `main` йшла першою. А щоб усе працювало, я розділив код на `.h` та `.cpp` файли, де в хедері є прототипи.
2. Якщо файлу немає, програма просто закриється з помилкою компіляції.
3. Щоб сума кожного рядка починалася з нуля, інакше туди додасться сміття з пам'яті.
4. `ios::app` дописує файл у кінець, а за замовчуванням файл затирається і створюється пустий.
5. Якщо знайдено дублікат, ми фізично видаляємо елемент з пам'яті і масив стає меншим на одну комірку.

ІІІ: Проаналізую твої відповіді.

Аналіз тестів:

- 1, 2, 3, 5 — абсолютно правильно.
4 — Помилка. Ти вказав Б ($O(N)$), але правильна відповідь В ($O(N^2)$).
Сортування вибором має вкладений цикл, який незалежно від стану масиву робить перевірки. Ймовірність використання ІІІ для цих відповідей: низька (є людська помилка).

Аналіз відкритих питань:

1. Правильно. Прототипи у .h файлі повідомляють компілятор про існування функцій до їх реалізації.
2. Помилка. Програма не видасть помилку компіляції (бо компіляція проходить раніше). Вона виведе повідомлення "Помилка відкриття файлу", встановить $n = 0$ і функція завершить роботу.
3. Абсолютно точно.
4. Правильно сформульовано.
5. Часткова помилка. У мові C++ статичний масив фізично не зменшується (комірки пам'яті залишаються). Відбувається "логічне" видалення: елементи просто зсуваються вліво, і ми зменшуємо лічильник n , щоб програма ігнорувала старий "хвіст" масиву.

Загальна оцінка: 3.8 / 5.

Я: Проаналізуй код у звіті, і додай опис і приклади коду з питань, які є в теоретичних відомостях, але не відпрацьовано у коді при вирішенні завдань.

ШІ: Аналізуючи код у звіті, можна помітити, що ти чудово відпрацював роботу з текстовими файлами (запис і читання чисел). Проте, у теоретичних відомостях часто згадується робота з бінарними файлами та читання посимвольно (наприклад, функцією `getline` з файлу), чого немає у коді лабораторної.

Приклад коду для зчитування файлу по рядках:

```
#include <fstream>
#include <iostream>
#include <string>
using namespace std;

void read_lines() {
    ifstream fin("text_data.txt");
    if(fin.is_open()) {
        string line;
        while(getline(fin, line)) {
            cout << line << endl;
        }
        fin.close();
    }
}
```